

DrowsyGuard: Secure Driver Drowsiness Detection

Course: Embedded Systems & IoT Security **Domain:** Driver drowsiness detection, an architectural clone of the example Fire Detector project retargeted to a different problem.

1. Introduction & Motivation

Driver fatigue is a well-known cause of road accidents. This project builds a real-time drowsiness detection system that runs on embedded hardware, raises a local alarm, and reports securely over the network, all without any cloud dependency.

Three constraints shaped every design decision in this project:

- **Runs on constrained hardware, in real time**, with no cloud dependency.
- **Secure:** telemetry leaving the device must be encrypted.
- **Testable:** this is safety-adjacent logic. "Trust me, it works" is not an acceptable standard. Every component must be independently verifiable without physical hardware.

2. Project Charter

Item	Content
Goal	Real-time system on a Raspberry Pi (emulated) that detects drowsiness via a CNN and triggers an alarm (buzzer + encrypted MQTT)
In scope	5 SRP components, TFLite model, mocked GPIO, AES security layer, Node-RED dashboard, adversarial attack/defense, TDD test suite, report + slides
Out of scope	Physical Raspberry Pi hardware, real camera on the Pi, cloud deployment, mobile app
Platform	Fedora + Python venv, fully emulated (except the Arduino companion, see §7)
Definition of Done	<code>python -m unittest</code> green on all components + end-to-end demo on video + report delivered

3. Requirements (frozen baseline)

Functional:

- FR1: acquire frames from a video source (continuous loop)
- FR2: preprocess frames (resize, color conversion, normalize 0–1)
- FR3: run CNN inference $\rightarrow P(\text{drowsy}) \in [0,1]$
- FR4: smooth over an N-frame window + apply a decision threshold
- FR5: activate the buzzer (PWM/GPIO) on drowsy detection, with hysteresis (minimum alarm duration)
- FR6: publish encrypted MQTT telemetry on detection
- FR7: display an overlay (probability, FPS, alarm state)

Non-functional:

- NFR1: runs without physical hardware (mock import fallback)
- NFR2: encrypted telemetry (AES-CBC, confidentiality)
- NFR3: testability, meaning controllability and observability via dependency injection and mocks
- NFR4: SRP, one responsibility per component

- NFR5: resilience: the video loops when it ends, and a broker outage does not block the pipeline (async)

4. Architecture

Five components, each with a single responsibility, wired together by dependency injection:

```
FrameProvider -> ImageProcessor -> InferenceEngine -> DrowsinessMonitor ->
AlertNotifier
```

`FrameProvider` handles video/webcam frame acquisition (`cv2.VideoCapture`) and loops back to the start when the video ends. `ImageProcessor` resizes frames, converts color, and applies the luminance transform the model was trained on (§6). `InferenceEngine` wraps the TFLite interpreter and returns `P(drowsy)`. `DrowsinessMonitor` is the orchestrator: it applies rolling-window smoothing and the threshold decision, driving the other four components. `AlertNotifier` handles the local buzzer (PWM/GPIO) and publishes encrypted MQTT telemetry, with alarm hysteresis.

No physical hardware is required to run the Raspberry Pi side: GPIO access goes through `mocks/GPIO.py` , with a `try: import RPi.GPIO / except: import mocks.GPIO` fallback used identically by every component that touches GPIO. This is what makes the whole system runnable and testable on a laptop.

5. Development Methodology (TDD)

Built strictly Red → Green → Refactor, component by component. Every component is tested against an injected mock, never a real dependency: `MockBuzzer` (Arduino side), mocked `cv2.VideoCapture` , mocked `tflite.Interpreter` , mocked `mqtt.Client` .

Test suite: 38 automated tests, all green.

- 31 Python tests (`unittest.mock`), run with `python -m unittest discover -s test`
- 7 Arduino/C++ tests (Unity, native PlatformIO environment, running entirely on the PC, no board required)

Coverage: measured with `coverage.py` , reaching 98% line coverage on `src/` (up from an initial 91%; see §9 for what the remaining 2% is and why a coverage-driven review found two real bugs).

Static analysis (flake8 and pylint) is part of the workflow, not an afterthought: every finding referenced in this report comes from actually running the tool, not from manual code reading. See §9 for the full methodology and findings log.

6. Model Pipeline

6.1 Preprocessing: a deliberate detour

Tried and abandoned: Haar-cascade eye cropping. The initial plan was to crop to the eye region before feeding the model, on the assumption that a tighter crop should help. This was abandoned for two concrete reasons:

- ~80% discard rate on this dataset (the cascade failed to detect an eye region in most samples), and even the "successful" detections were sometimes false positives.
- A hard blocker: `cv2.CascadeClassifier.detectMultiScale()` segfaults when TensorFlow is imported in the same process on the training machine, confirmed via isolated reproduction,

independent of `opencv-python` vs. `opencv-python-headless`, independent of import order, and independent of `cv2.setNumThreads(1)`. A real native library conflict, not a fluke.

Final approach: no crop, no cascade, no filtering. Full frame, resized to a fixed 96×96 square, converted into a custom 3-channel tensor:

- **Y**: luminance (from `cv2.COLOR_RGB2YCrCb`, channel 0)
- **B**: raw blue channel from the original RGB image
- **R**: raw red channel from the original RGB image

This design was forced by a real constraint, not an aesthetic choice: the GPU training backend (Intel ITEX/oneDNN) does not support single-channel Conv2D input, so 3 channels were mandatory regardless. Since the channel count, not its content, determines parameter count, the third channel carries real signal (raw R) instead of a duplicated Y.

6.2 Model architecture

The model is a custom CNN with 341,121 parameters, comfortably under a 2M-parameter budget, built from scratch with no transfer learning. This is a deliberate choice, and it deviates from the fire-detector template's MobileNetV2 approach, in order to keep the model lightweight. Training used the HuggingFace `driver-drowsiness-dataset` (binary, `drowsy` / `not_drowsy`): 18,492 train, 2,311 validation, and 2,313 test images. The splits were kept strictly separate on disk and never re-merged or reshuffled, because the dataset already contains augmented variants, and re-splitting risks the same source image ending up in both train and test (data leakage).

One counter-intuitive finding: adding `class_weight='balanced'` to counter a 1.3:1 class imbalance in the training set made results worse, not better. The recall gap widened from 0.61/0.89 to 0.51/0.91. The likely cause is an interaction with early stopping's loss-based patience: a weighted loss is a noisier stopping signal. Final configuration: `USE_CLASS_WEIGHT = False`.

6.3 Results, reported honestly

	precision	recall	f1-score	support
not_drowsy	0.710	0.751	0.730	1007
drowsy	0.799	0.764	0.781	1304
accuracy			0.758	2311
macro avg	0.755	0.757	0.756	2311

Accuracy is 75.8%, below the original 85% target. This number is presented as-is rather than rounded up or hidden. Training was deliberately not resumed to chase a higher number. This was an explicit scope decision (see §11, "will not fix") given time constraints, not an oversight. What is emphasized instead is that recall is fairly balanced across both classes (0.751 vs. 0.764), which for a safety alarm matters more than a higher accuracy achieved by being much better at one class than the other.

7. Security

7.1 Encrypted MQTT telemetry

On drowsy detection, `AlertNotifier` does not only drive the local buzzer. It also publishes an encrypted telemetry payload:

```
AlertNotifier -> AES-256-CBC encrypt -> Mosquitto broker -> Node-RED dashboard
```

Every payload is AES-256-CBC encrypted (`pycryptodome`) before publish, with a random IV per message, verified by a dedicated test (`test_encrypt_data_uses_random_iv_each_time`). Cross-language interoperability was verified against real payloads, not just unit-tested in isolation: encrypted in Python, decrypted in JavaScript inside Node-RED (`crypto.createDecipheriv`), with the AES key injected via an environment variable rather than hardcoded. The dashboard itself runs on Node-RED plus `node-red-dashboard` , with a live gauge for P(drowsy), a status text field, and a history chart, served at `localhost:1880/ui` alongside Mosquitto via `docker-compose.yml` .

7.2 Adversarial attack and defense

To demonstrate a real vulnerability against the trained model, two things were built:

- **Attack:** PGD-style gradient ascent on a 20×20 pixel patch, 50 steps, optimizing pixel values to maximize the model's loss (`epsilon=8.0`).
- **Defense:** Gaussian blur input sanitization, requiring no retraining.

Stage	P(drowsy)	Correct?
Baseline	0.41	✓ (not drowsy)
Patched (attack)	0.998	× (flipped to "drowsy")
Sanitized (defense)	0.48	✓ (restored)

Starting from a genuinely not-drowsy sample, correctly classified at 0.41, the optimized patch pushes the model's confidence to 0.998: a confident, wrong "drowsy" verdict on a clean sample. A simple Gaussian blur applied as input sanitization, with no retraining, restores the correct prediction.

8. Arduino Hardware Companion

The Raspberry Pi side of this project runs entirely emulated: no physical Pi, GPIO mocked throughout, by design (§4). To have a genuinely physical alarm, a companion device was built, an Arduino Uno R4 WiFi that subscribes to the same encrypted MQTT topic and reacts to it on real hardware.

```
MQTT receive -> AES-256-CBC decrypt (on-device) -> TelemetryHandler -> alarm
```

`tiny-AES-c` (vendored, forced to AES-256 via a `-DAES256=1` build flag) decrypts the payload directly on the board. `TelemetryHandler` mirrors `AlertNotifier.notify()` 's hysteresis logic in C++, adapted to the fact that the Arduino only ever receives "alarm start" MQTT messages (Python never publishes an explicit "stop," see §7.1), so the handler times itself out via `update()` , polled in the main loop. Since no physical buzzer is wired yet, `RealBuzzer::start() / stop()` also drive the board's built-in 12×8 monochrome LED matrix (all on / clear) as a temporary visual stand-in, marked explicitly as such in the code and in this report, to be swapped back to a real buzzer once one is wired. The firmware has two PlatformIO environments: `native` for Unity tests on the PC (no hardware required) and `uno_r4_wifi` for the real board (`renesas-ra` platform, WiFiS3 + ArduinoMqttClient).

8.1 Three real bugs, found only on real hardware

None of these three bugs showed up in the unit test suite. All three only surfaced once the system ran end to end on physical hardware and real network conditions.

1. **Wrong WiFi SSID.** Diagnosed by adding a temporary `WiFi.scanNetworks()` dump to `main.cpp`. The board had zero serial output before this; no logging existed at all, so debugging was completely blind until it was added.
2. **LED matrix never lit**, despite MQTT → AES → `TelemetryHandler` all confirmed working correctly in the logs. The root cause: `RealBuzzer` is a file-scope global object, so its constructor runs during C++ static initialization, which happens before the board's own `init()` (clocks/peripherals). `ArduinoLEDMatrix::begin()` allocates a hardware timer for the LED-refresh interrupt; called that early, the allocation silently fails. `loadFrame()` still "succeeds" in software, but nothing physically lights up, and there is no error anywhere. Fixed by moving hardware init out of the constructor into an explicit `begin()`, called from `setup()` once the board is actually ready.
3. **Model never triggered on live webcam**, even with eyes closed. The root cause: live inference was feeding plain normalized RGB, while the model was trained on the Y/B/R transform (§6.1), a preprocessing pipeline mismatch invisible to unit tests, because no test in the suite exercises a real camera frame end to end. Fixed by adding the equivalent Y/B/R transform to `ImageProcessor`.

8.2 End-to-end verification and a known limitation

The full loop was verified live: webcam feed → smoothing/threshold logic → encrypted MQTT → physical Arduino → LED matrix lights on detection.

Known limitation, documented as a deployment constraint, not a bug: the model only reacts reliably when the driver's face fills most of the frame, confirmed empirically (close to the camera, it works; at normal webcam distance, confidence stays flat around 0.10–0.14, essentially insensitive). The root cause is that the training dataset consists of close-up face crops; at 96×96, a small or distant face loses essentially all eye detail. Automatic face-cropping was deliberately not added to fix this: that path was already tried and abandoned during training for the TensorFlow/cascade segfault reason in §6.1, and the same conflict risk applies live, since `InferenceEngine` still imports `tensorflow.lite` in-process. It is documented instead as a hardware constraint: this system is designed for a dashboard-mounted, close-range camera, like a real driver-monitoring system, not a general-purpose room webcam.

9. Testing & Quality Assurance

9.1 Coverage-driven bug hunt

Beyond the TDD suite built alongside each component, a dedicated review pass was done late in the project specifically to answer: how much of the code is actually tested, and what happens on the paths that aren't? `coverage.py` was installed to get real numbers instead of guessing, and every uncovered line was read and reasoned about individually, not just treated as a percentage to close mechanically.

Result: line coverage on `src/` went from 91% to 98%. New tests closed real behavioral gaps that had been silently untested:

- `AlarmState` hysteresis was only tested for "enough time has passed, clear the alarm." The symmetric case (not enough time has passed, so the alarm should stay active) was untested.
- `notify(drowsy_detected=False)` when the alarm was never triggered (a realistic case: the system starts and the driver is never drowsy) was untested.
- `_on_connect` / `_on_disconnect` (the paho-mqtt callbacks) were 0% covered. The MQTT client is mocked everywhere else, so these callbacks were registered but never actually invoked by any test, and nothing proved `_connected` updates correctly on the real callback contract (`rc==0` or not).
- The `publish_via_mqtt` guard for "not connected, skip publish" was untested; nothing proved the local alarm still fires correctly while the broker is unreachable.

- `FrameProvider` 's double-read-failure edge case (video ends, rewind attempted, rewind also fails) was untested.
- The interactive 'q'-to-quit key path in `DrowsinessMonitor.run()` was untested.

One gap was consciously left open: the `GPIOMOCK = False` branch (the real `RPi.GPIO` import succeeding) cannot be exercised without physical Raspberry Pi hardware, accepted as an intrinsic limitation of the emulated-hardware approach (§4), not a process failure.

This review surfaced two real bugs, not just missing tests.

1. **`AlertNotifier.cleanup()` was never called anywhere in production code.** The method exists (it stops the buzzer, releases GPIO, and stops the MQTT client loop), but `DrowsinessMonitor._cleanup()` only called `cv2.destroyAllWindows()`. On real hardware, this meant GPIO pins were never released on shutdown. Proven with a failing test (`AssertionError: Expected 'cleanup' to have been called once. Called 0 times.`) before the one-line fix was applied.
2. **Arduino `TelemetryHandler::onMessage()` crashed on malformed input.** `const char* status = jdoc["status"];` returns `nullptr` when valid JSON is missing the `status` key, and the following `strcmp(status, ...)` call is then undefined behavior on a null pointer. This is not reachable in normal operation, since the Python side always sends `status`, but it is a real robustness gap against malformed or corrupted input reaching the decrypt step. Proven with an actual SIGSEGV (segmentation fault) on the native test build, not just theoretical UB, before a null-check fix was applied.

9.2 Static analysis and refactoring

flake8 is 100% clean on `src/`. pylint flagged a real `too-many-instance-attributes` (R0902) finding on `AlertNotifier` (10 attributes, threshold 7), resolved by extracting two collaborator classes via composition:

- `AlarmState` : the alarm start/stop hysteresis logic, a genuinely cohesive, independently testable block of behavior, not just a data grouping.
- `MqttConfig` : a lightweight value object grouping the MQTT connection parameters.

This brought `AlertNotifier` from 10 down to 6 attributes, resolving the finding (file-isolated pylint score 8.55/10 afterward).

`DrowsinessMonitor` was deliberately left unrefactored, despite triggering the same R0902 warning (8 attributes, only 1 over threshold). This was a conscious decision, not an oversight: of those 8 attributes, 4 are already-correct dependency-injected collaborators (`frame_provider`, `image_processor`, `inference_engine`, `alert_notifier`), each with its own single responsibility and its own test suite via mocks. pylint's attribute count does not distinguish "wiring of injected dependencies" from "real internal state." Refactoring this class further would only have quieted the linter, not fixed an actual design problem, at the cost of extra indirection with no real cohesion gain.

Every static analysis finding across the project's history is logged, with what was found, by which tool, and who applied the fix (see `IMPLEMENTATION_PLAN.md` §8 for the full, commit-by-commit log). One known, accepted gap: `test/*.py` was never itself brought to flake8 compliance (only `src/` was). 22 pre-existing findings (mostly line length) were identified but deliberately left unaddressed, flagged rather than silently fixed, as a separate task outside this project's time budget.

10. Use of AI in This Project

This project was built collaboratively with an AI assistant, under a consistent set of ground rules kept throughout: the assistant never writes `src/*.py` or the tested Arduino core logic directly, that code is

authored by the author of this report. The assistant's role was scoped as follows:

- **Planning help:** structuring an effective work plan.
- **Autonomous:** reviewing the tests.
- **Autonomous:** reviewing code smells and other static-analysis tool findings.
- **Autonomous for low-hanging fruit, manual for complex cases:** patching findings. Mechanical fixes (dead imports, formatting, missing newlines) were applied directly; anything involving a real design decision, such as the `AlertNotifier` composition refactor or the Arduino null-check fix, was implemented by hand, after the assistant identified and explained the issue.
- **Autonomous:** writing commits and pushing after a prompt, and updating progress documentation (`IMPLEMENTATION_PLAN.md`) to reflect completed steps.
- **Autonomous:** reviewing code to find hidden bugs and untested edge cases, with a subsequent suggestion of the tests needed to close those gaps (see §9.1 for the concrete results of this).

Autonomous here means done by the AI after a prompt, without step-by-step supervision. *Manual* means done by the author, after the AI pointed out the issue. Scripts and tools not covered by the graded test suite, such as `train_model.py` , `prepare_dataset.py` , `security/adversarial_patch.py` , `main.py` , and the hardware smoke-test tooling, were written directly by the assistant, as an explicit, narrower exception to the rule above.

11. Deliberate Technical Debt

Everything in this section is a conscious, scoped decision, not an oversight. It is flagged explicitly here rather than left implicit, so it reads as "known and accepted" rather than "missed."

- **Missing docstrings.** pylint reports 21 real findings (`C0114` / `C0115` / `C0116` , missing module/class/function docstrings) across `src/*.py` and `main.py` (verified with `pylint src/main.py`). Left unaddressed by choice: this project is already documented extensively at the architecture and decision level, in this report, in the implementation plan, and in inline comments that explain why something is done a certain way, not what it does, which well-named functions and classes already communicate on their own. Adding a docstring to every method with an already-self-explanatory name would be redundant boilerplate, not new information. The time this would have taken went instead into the coverage-driven review in §9.1, which found two real bugs, judged the better use of limited remaining time.
- **Model accuracy (75.8%, target was >85%): will not fix.** Training was paused as a deliberate checkpoint; resuming it to chase a higher number was not judged worth the remaining time. The result is reported honestly in §6.3 rather than hidden or rounded up.
- **Recorded demo video clips: will not fix.** The project's frozen Definition of Done (§2) requires an end-to-end demo on video, which is already satisfied by the live webcam-to-Arduino run described in §8.2. A separately recorded clip would only add supporting evidence for a presentation, not close a requirement gap, and was not worth the remaining time.
- **test/*.py flake8 compliance:** 22 pre-existing findings (§9.2), flagged, not fixed.
- **Physical Raspberry Pi hardware and real secrets.h /camera deployment:** explicitly out of scope for this project from the start (§2). Only the Arduino companion was targeted at real hardware, and that hardware bring-up (§8) was completed and verified.

12. Conclusion

DrowsyGuard delivers a complete, tested, secure driver-drowsiness detection pipeline: five single-responsibility components built test-first, a lightweight custom CNN with honestly reported results, AES-256-CBC encrypted telemetry with a demonstrated adversarial attack and a working defense, and a physical Arduino companion that surfaced three real hardware bugs no unit test could have caught. A later, deliberate

coverage-driven review of the test suite itself, going from 91% to 98% line coverage on `src/`, found two further real bugs, one of them reproduced as an actual crash, not just a theoretical risk. What remains open is reported transparently rather than hidden: a below-target model accuracy, accepted under time constraints, and a small, explicitly scoped set of "will not fix" items.